

LyDROO: 큐와 전력 제약을 함께 보는 MEC 오프로딩

Lyapunov-Guided Deep Reinforcement Learning for Stable Online
Computation Offloading in Mobile-Edge Computing Networks

Suzhi Bi, Liang Huang, Hui Wang, Ying-Jun Angela Zhang (2021)

한 frame의 빠른 선택을 장기 queue 안정성과 평균 power 제약으로 연결하는 hybrid online controller

좋은 channel의 한 frame 선택만으로는 충분하지 않다

현재 rate만 크게 만들면 average power를 넘기거나 task를 뒤로 미뤄 queue를 키울 수 있다

- | | | |
|----|--------------|-----------------------------------|
| 01 | 현재 관측 | current channel h 와 Q, Y 를 본다 |
| ↓ | | |
| 02 | 즉시 rate 최대화 | 좋은 channel에서 energy를 많이 쓸 수 있다 |
| ↓ | | |
| 03 | 누적 효과 | 평균 power 또는 대기 task가 쌓인다 |
| ↓ | | |
| 04 | 필요한 feedback | Q 와 Y 를 다음 frame 의사결정에 반영 |

LyDROO의 목적은 frame reward가 아니라 장기 queue와 평균 power를 함께 제어하는 것이다

한 device의 한 frame: route와 연속 resource를 분리한다

binary x 가 local/offload route를 고르고, route 안의 f 또는 $\tau \cdot e$ 가 실제 service를 정한다



DNN은 final CPU frequency나 frame power를 직접 내보내지 않는다

Q는 실제 workload, Y는 average-power 압력 기록이다

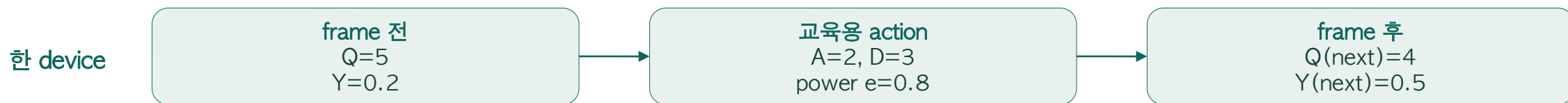
둘 다 다음 action을 바꾸지만 물리적 의미는 다르다

state	update	무엇을 뜻하나	커지면
Q_i	$Q(\text{next})=Q-D+A$	아직 service되지 않은 task data	현재 service의 가치가 커짐
Y_i	$Y(\text{next})=\max(Y+\eta(e-\gamma),0)$	average-power 초과 압력의 virtual record	power 사용의 비용이 커짐
A, D	arrival, served data	frame에서 들어오고 처리된 data	D는 Q를 넘을 수 없음
e, γ	power, threshold	frame power와 장기 average-power threshold	e 가 γ 보다 크면 Y 증가

Y는 battery charge가 아니라 장기 average-power constraint를 online으로 보이는 queue다

작은 queue trace: Q와 Y의 다른 변화

교육용 계산: $\gamma=0.5$, $\nu=1$. 논문의 측정값이 아니다



D=3은 workload를 줄였지만 e- $\gamma=0.3$ 은 다음 frame의 power pressure를 높인다

Lyapunov objective: service와 power pressure

P2와 그 P3/P4 분해는 action과 무관한 drift 항을 빼고 current state가 가중한 목표를 푼다

항	읽는 법	state가 커질 때	결정 효과
$(Q_i + V c_i) r_i$	service rate의 보상	Q_i 가 크면 증가	waiting task를 더 service하려 함
$Y_i e_i$	power consumption penalty	Y_i 가 크면 증가	power-heavy action을 억제
V	utility와 queue의 control parameter	V 가 크면 utility weight 증가	utility gap와 backlog의 trade-off
constraint	binary route와 continuous resource feasibility	τ 합과 data causality	각 frame feasible action만 비교

Q 와 Y 는 history를 저장해 다음 frame의 one-frame objective를 시간-aware하게 만든다

V trade-off: 정리의 bound와 Figure 7

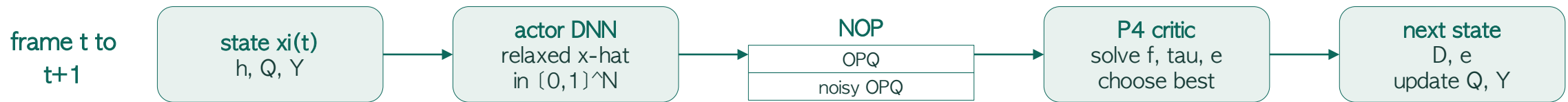
정리는 조건부 rate-gap/queue bound를 주고, Figure 7은 setup-dependent한 non-monotone 관측을 보인다

V	objective에서	정리의 방향	실험에서 읽을 점
작은 V	Q의 상대 영향이 큼	rate-gap bound는 더 큼	Figure 7에서 $V \leq 40$ 의 Q/power 감소
큰 V	utility c의 상대 영향이 큼	rate-gap bound 감소, queue bound 증가	Figure 7에서 $V > 40$ 의 Q/power/Y 증가
주의	control parameter	assumptions와 bounded gap 아래	rate curve의 단조 증가 증명 아님

V의 bound trade-off는 조건부다. Figure 7을 단조 tuning rule로 읽으면 안 된다

LyDROO online loop

queue module의 Q/Y가 다음 frame actor input으로 다시 들어간다



DNN이 binary search를 좁히고 P4가 continuous feasibility와 value를 판정한다

Actor와 critic은 모두 neural network가 아니다

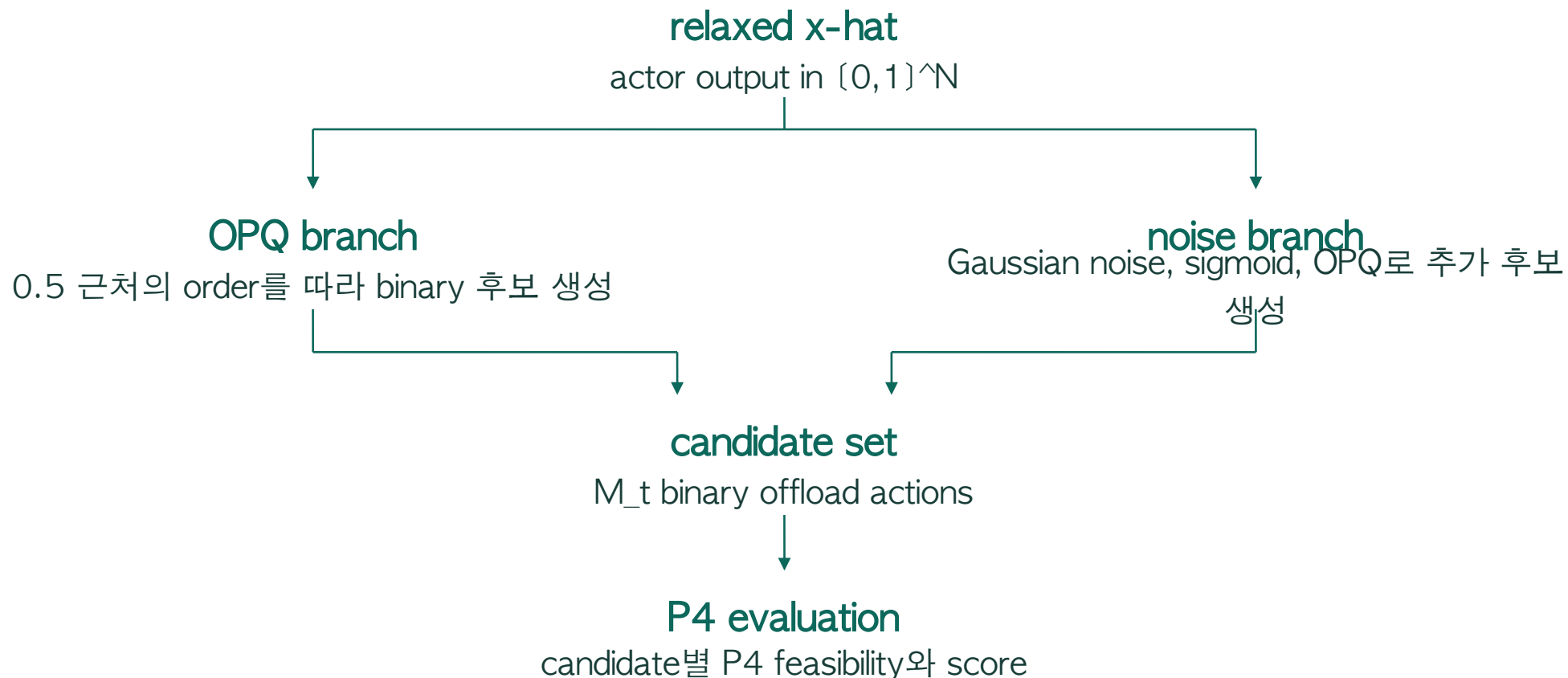
LyDROO의 critic은 learned value estimator가 아니라 candidate마다 P4를 푸는 analytical optimizer다

01	actor DNN	state에서 relaxed binary proposal을 낸다
↓		
02	NOP	proposal 주변의 binary actions를 만든다
↓		
03	critic P4	candidate마다 continuous allocation을 계산한다
↓		
04	argmax	modeled objective가 가장 큰 feasible action을 실행
↓		
05	training label	선택된 action이 actor training target이 된다

‘actor-critic DRL’이라는 이름을 일반적인 learned-critic 구조로 읽으면 안 된다

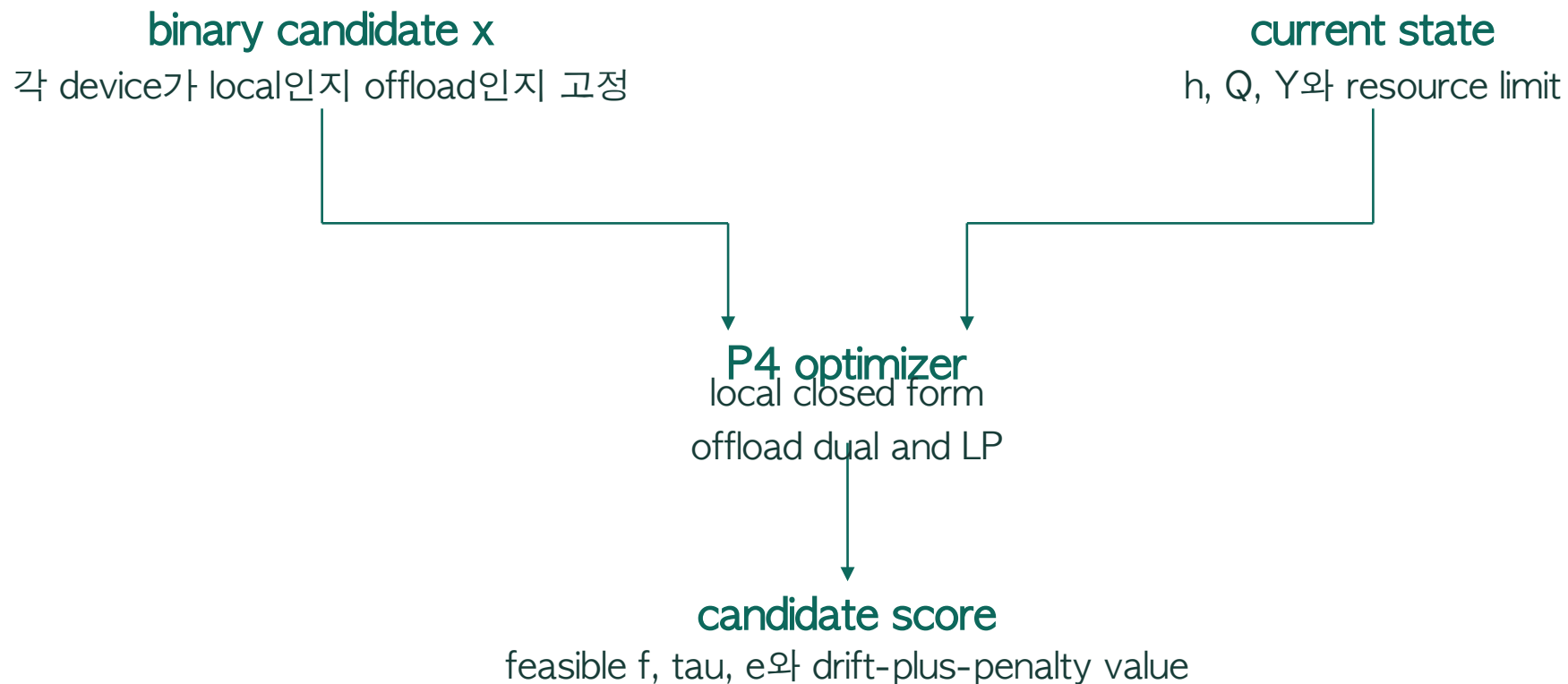
NOP: actor 후보와 noise 후보

order-preserving quantization이 exploitation을, noisy branch가 exploration을 담당한다



P4는 binary candidate에 조건부로 연속 resource를 푼다

같은 x 라도 Q , Y , h 가 달라지면 best f , τ , e 와 objective가 달라진다



Online action과 actor training의 시간축

매 frame에는 candidate evaluation을 하고, DNN parameter update는 replay memory에서 주기적으로 수행한다

과정	입력	언제	출력
online control	current h , Q , Y	매 frame	x , f , τ , e
critic evaluation	binary candidates와 current state	online control 안	P4 objective와 best label
replay storage	selected state-action pair	action 뒤	최근 q 개 sample
actor training	random replay batch	δ_{T-1} frame마다	다음 actor parameter

reported CPU computation time은 online controller 측정치이며 training 시간 전체가 아니다

Theorem 1 : 조건부 whole-policy 결과

stability와 trade-off는 stochastic model, feasibility, 그리고 bounded approximation gap을 함께 가정한다

필요한 조건	왜 필요한가	정리가 말하는 것	말하지 않는 것
bounded arrival와 service	drift bound를 만들기 위함	average queue bound	무한 burst에도 자동 안정
i.i.d. system state	time-average expectation 처리	stochastic guarantee	모든 non-i.i.d. deployment
feasibility와 Slater-type 조건	constraint를 만족할 여유	virtual queue stability	불가능한 budget의 해결
bounded per-frame gap C	selected action이 ideal P2에서 너무 멀지 않음	$O(1/V)$, $O(V)$ trade-off	임의 training run의 성공

Theorem 1은 approximation gap을 포함한 controller 전체를 조건부로 분석한다

Queue plot은 defined simulation에서의 비교 증거다

Figure 5의 $\lambda=3$ run에서 Myopic queue는 선형으로 자라고 LyDROO는 LyCD에 접근한다

setting	reported observation	올바른 해석
$N=3, \lambda=3$	DDPG baseline은 stabilize하지 못함	textual simulation observation
Figure 5, $\lambda=3$	Myopic queue가 linear growth	frame-only choice의 failure mode 예시
Figure 5, LyDROO	약 7,500 frame 뒤 LyCD에 접근	한 simulation trace의 timing
Figure 6(a), $\gamma=0.08$	displayed λ range에서 3.2까지 stable	universal threshold가 아님

queue plot은 지정된 workload와 parameter에서 얻은 empirical evidence다

Figure 7: V 와 queue/power의 non-monotone 관측

converged setting에서 rate는 같고, queue·power는 V 구간에 따라 다른 방향을 보인다

관측 대상	plot에서의 방향	정리와의 연결	한계
$V \leq 40$	queue와 power 감소	theorem bound 자체의 plot 아님	converged simulation observation
$V > 40$	queue, power, energy queue 증가	queue bound 방향과 양립	exact threshold는 setup-dependent
computation rate	LyDROO와 LyCD의 optimal performance	rate-gap bound와 구분	rate의 단조 증가 claim 아님

Figure 7은 setup-specific observation이다. V 의 theorem bound와 구분해야 한다

N=30 reported CPU computation time의 범위

paper의 0.156 s는 TensorFlow 2, Intel i5, 12 GB RAM 환경의 Table II 수치다

조건	reported number	paper의 비교	피해야 할 해석
N=30	reported CPU computation time 0.156 s	5-s frame의 약 3%	generic edge-device latency
same table	LyCD보다 약 50배 빠름	defined comparator implementation	모든 MEC solver의 배수
adaptive M_t	candidate selection runtime 80% 이상 절감	cited comparison의 saving	무조건 80% cost reduction
environment	TensorFlow 2, Intel i5, 12 GB RAM	simulation/controller setting	mobile hardware benchmark

reported CPU time은 paper의 online controller 측정치다

Figure 9: non-i.i.d. test와 theorem의 구분

N=10 ON-OFF Markov arrival test는 empirical evidence다. Theorem 1의 i.i.d. bound와 구분한다

부분	paper가 한 일	근거 강도	구분
Theorem 1	i.i.d. $\omega(t)$ 를 가정	조건부 queue/rate bound	일반 non-i.i.d. proof 아님
Figure 9 test	N=10 ON-OFF Markov와 i.i.d. exponential, 평균 3 Mbps	empirical: Markov에서 더 긴 convergence	plot 자체가 theorem 증명 아님

Figure 9의 non-i.i.d. result와 Theorem 1의 i.i.d. bound는 근거 종류가 다르다

DROO와 LyDROO의 control-state 차이

LyDROO는 queue와 average-power constraint를 current state와 objective에 추가한다

dimension	DROO formulation	LyDROO formulation
actor input	wireless channel h	channel h 와 real Q , virtual Y
per-frame aim	computation rate	drift-plus-penalty objective
candidate generation	relaxed output의 OPQ	OPQ와 noisy OPQ, NOP
continuous solver	fixed- x convex allocation (P2)	Lyapunov-weighted P4
long-term feedback	Q/Y control state 없음	Q, Y update가 next action을 바꿈

LyDROO를 DROO에 reward 하나를 더한 방법으로 축소하면 Q/Y feedback loop를 놓친다

모델의 경계도 controller 해석의 일부다

analytical critic의 feasible action과 theorem claim은 paper가 둔 communication·energy model 안에서만 읽어야 한다

모델 선택	paper의 처리	해석 영향
offloading	binary local 또는 edge route	local/edge split 없음
uplink	TDMA, $\sum \tau \leq 1$	shared time coupling
edge stage	execution과 download delay 무시	end-to-end latency 전체가 아님
utility	linear computation rate	다른 U는 convex P4가 필요
queue	infinite capacity for analysis	finite buffer drop model과 다름

해석적 critic은 일반 oracle이 아니라 이 논문의 MEC model을 푸는 optimizer다

LyDROO를 읽는 네 가지 연결

queue update가 objective를 바꾸고, actor proposal이 analytical allocation과 결합되며, theorem과 plot는 다른 근거를 제공한다

- | | | |
|----|----------------------|--|
| 01 | Q와 Y | workload와 average-power pressure를 state로 보존 |
| ↓ | | |
| 02 | drift plus penalty | service reward와 power penalty를 current state로 가중 |
| ↓ | | |
| 03 | actor plus P4 | binary search를 좁히고 continuous feasibility를 계산 |
| ↓ | | |
| 04 | conditional evidence | theorem assumptions와 simulation conditions를 분리 |

LyDROO는 long-term feedback와 model-based candidate evaluation을 결합한다